



UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ciencias

Escuela Profesional de Ciencia de la Computación

EXAMEN PARCIAL

Curso: **Fundamentos de Programación CC112 - A**

Nombres y Apellidos: ELIAZ SEBASTIAN BOBADILLA CAMARENA

Código: 20244697E (c++)

```

obrima_ano_vector.hpp / PrintArray(int *, int, ostream &)
#include <iostream>

using namespace std;

void intercambiar(int &a, int &b)
{
    int temp = a;
    a = b;
    b = temp;
}

void PrintArray(int *vec, int tam, ostream &out)
{
    for (int i = 0; i < tam; ++i)
    {
        out << vec[i] << " ";
    }
    out << endl;
}

void DestroyVector(int *&vec)
{
    delete[] vec;
    vec = nullptr;
}

void CreateVector(int *&vec, int tam)
{
    vec = new int[tam];
}

void ReadArray(int *vec, int tam, istream &in)
{
    for (int i = 0; i < tam; ++i)
    {
        in >> vec[i];
    }
}

void Intercambiar(int *&a, int *&b)
{
    int *temp = a;
    a = b;
    b = temp;
}

// ELIAZ SEBASTIAN BOBADILLA CAMARENA 20244697E

```

```

// Función para ordenamiento de burbuja recursivo ascendente
void BurbujaRecursivoAscendente(int *arr, int n)
{
    if (n <= 1)
        return;
    for (int j = 1; j < n; ++j)
    {
        if (arr[j] > arr[0])
        {
            intercambiar(arr[0], arr[j]);
        }
    }
    BurbujaRecursivoAscendente(arr + 1, n - 1);
}

// Función para ordenamiento de burbuja recursivo descendente
void BurbujaRecursivoDescendente(int *arr, int n)
{
    if (n <= 1)
        return;
    for (int j = 1; j < n; ++j)
    {
        if (arr[j] < arr[0])
        {
            intercambiar(arr[0], arr[j]);
        }
    }
    BurbujaRecursivoDescendente(arr + 1, n - 1);
}

int main()
{
    int *vec1 = nullptr, size = 10;
    int *vec2 = nullptr;

    CreateVector(vec1, size);
    ReadArray(vec1, size, cin);
    BurbujaRecursivoAscendente(vec1, size); // Ascendente
    PrintArray(vec1, size, cout);

    CreateVector(vec2, size);
    ReadArray(vec2, size, cin);
    BurbujaRecursivoDescendente(vec2, size); // Descendente
    PrintArray(vec2, size, cout);

    Intercambiar(vec1, vec2); // Intercambiar punteros

    DestroyVector(vec1);
    DestroyVector(vec2);
}

```

```
• → maxe g++ problema_uno_vector.cpp
• → maxe ls
a.out      problema_dos_determinantes.cpp problema_uno_vector
problema_cuatro_cadenas.cpp  problema_tres_mapa.cpp  problema_uno_vector.cpp
```

DOS

```

#include <iostream>

using namespace std;

double valorasabsoluto(double x)
{
    return x * ((x > 0) - (x < 0));
}

int redondeador(double x)
{
    if (x >= 0.0)
        return static_cast<int>(x + 0.5);
    else
        return static_cast<int>(x - 0.5);
}

double determinant(const double *A, int n)
{
    if (n == 1)
    {
        return A[0];
    }

    int bestRow = 0, maxZeros = -1;
    for (int i = 0; i < n; ++i)
    {
        int zeroCount = 0;
        for (int j = 0; j < n; ++j)
        {
            if (A[i * n + j] == 0.0)
                ++zeroCount;
        }
        if (zeroCount == n)
        {
            return 0.0;
        }
        if (zeroCount > maxZeros)
        {
            maxZeros = zeroCount;
            bestRow = i;
        }
    }

    double det = 0.0;
    double *sub = new double[(n - 1) * (n - 1)];
    for (int i = 0; i < n; ++i)
    {
        if (i == bestRow)
            continue;
        for (int j = 0; j < n; ++j)
        {
            if (j == bestRow)
                continue;
            sub[j * (n - 1) + i] = A[j * n + i];
        }
    }
    det = A[bestRow * n] * determinant(sub, n - 1);
    for (int i = 0; i < n; ++i)
    {
        if (i == bestRow)
            continue;
        det += (-1)^(i + bestRow) * A[i * n] * determinant(sub, n - 1);
    }
    delete[] sub;
    return det;
}

int main()
{
    int n;
    double *A;
    cin >> n;
    A = new double[n * n];
    for (int i = 0; i < n; ++i)
    {
        for (int j = 0; j < n; ++j)
        {
            cin >> A[i * n + j];
        }
    }
    double det = determinant(A, n);
    cout << det << endl;
    return 0;
}

```

```

48     for (int j = 0; j < n; ++j)
49     {
50         double val = A[bestRow * n + j];
51         if (val == 0.0)
52             continue;
53
54         for (int r = 0, si = 0; r < n; ++r)
55         {
56             if (r == bestRow)
57                 continue;
58             for (int c = 0, sj = 0; c < n; ++c)
59             {
60                 if (c == j)
61                     continue;
62                 sub[si * (n - 1) + sj] = A[r * n + c];
63                 ++sj;
64             }
65             ++si;
66         }
67
68         double sign = ((bestRow + j) % 2 == 0) ? 1.0 : -1.0;
69         det += sign * val * determinant(sub, n - 1);
70     }
71
72     delete[] sub;
73     return det;
74 }
75 // ELIAZ SEBASTIAN BOBADILLA CAMARENA 20244697E
76
77 int main()
78 {
79     int n;
80     cout << "Ingrese el tamaño de la matriz (máximo 10): ";
81     if (!(cin >> n) || n < 1 || n > 10)
82     {
83         cout << "Tamaño inválido. Debe ser entre 1 y 10.\n";
84         return 1;
85     }
86
87     double *A = new double[n * n];
88     cout << "Ingrese los " << (n * n) << " elementos de la matriz:\n";
89     for (int i = 0; i < n * n; ++i)
90     {
91         cin >> A[i];
92     }
93
94     double det = determinant(A, n);
95     double absDiff = valorabsoluto(det - redondeador(det));

```

```

75 // ELIAZ SEBASTIAN BOBADILLA CAMARENA 20244697E
76
77 int main()
78 {
79     int n;
80     cout << "Ingrese el tamaño de la matriz (máximo 10): ";
81     if (!(cin >> n) || n < 1 || n > 10)
82     {
83         cout << "Tamaño inválido. Debe ser entre 1 y 10.\n";
84         return 1;
85     }
86
87     double *A = new double[n * n];
88     cout << "Ingrese los " << (n * n) << " elementos de la matriz:\n";
89     for (int i = 0; i < n * n; ++i)
90     {
91         cin >> A[i];
92     }
93
94     double det = determinant(A, n);
95     double absDiff = valorasbsoluto(det - redondeador(det));
96     if (absDiff < 1e-9)
97     {
98         cout << "Determinante = " << redondeador(det) << "\n";
99     }
100    else
101    {
102        cout << "Determinante = " << det << "\n";
103    }
104
105    delete[] A;
106    return 0;
107 }
108

```

```

● → maxe g++ problema_dos_determinantes.cpp
○ → maxe ./problema_uno_vector
● → maxe ls
a.out          problema_dos_determinantes.cpp problema_uno_vector
problema_cuatro_cadenas.cpp problema_tres_mapa.cpp problema_uno_vector.cpp
● → maxe ./a.out
Ingrese el tamaño de la matriz (máximo 10): 3
Ingrese los 9 elementos de la matriz:
1 0 0
2 4 5
5 7 3
Determinante = -23

```

PROBLEMA 3

```

1  #include <iostream>
2  using namespace std;
3
4  const int MAX = 100;
5
6  void mappa(int **mapa, bool **visitado, int filas, int columnas, int x, int y)
7  {
8      if (x < 0 || x >= filas || y >= columnas || visitado[x][y] || mapa[x][y] == 1 || y < 0)
9      {
10         return;
11     }
12
13     visitado[x][y] = true;
14
15     mappa(mapa, visitado, filas, columnas, x - 1, y);
16     mappa(mapa, visitado, filas, columnas, x + 1, y);
17     mappa(mapa, visitado, filas, columnas, x, y - 1);
18     mappa(mapa, visitado, filas, columnas, x, y + 1);
19 }
20
21 int main()
22 {
23     int filas, columnas;
24     cout << "Ingrese el número de filas y columnas del mapa: ";
25     cin >> filas >> columnas;
26
27     if (filas > MAX || columnas > MAX)
28     {
29         cout << "Dimensiones exceden el máximo permitido de " << MAX << "x" << MAX << endl;
30         return 1;
31     }
32
33     int **mapa = new int *[filas];
34     for (int i = 0; i < filas; ++i)
35     {
36         mapa[i] = new int[columnas];
37     }
38
39     bool **visitado = new bool *[filas];
40     for (int i = 0; i < filas; ++i)
41     {
42         visitado[i] = new bool[columnas];
43         for (int j = 0; j < columnas; ++j)
44             visitado[i][j] = false;
45     }
46 }

```



```

47     cout << "Ingrese los valores del mapa (0 = camino, 1 = obstáculo):\n";
48     for (int i = 0; i < filas; ++i)
49     {
50         for (int j = 0; j < columnas; ++j)
51         {
52             cin >> mapa[i][j];
53         }
54     }
55
56     int regiones = 0;
57
58     for (int i = 0; i < filas; ++i)
59     {
60         for (int j = 0; j < columnas; ++j)
61         {
62             if (mapa[i][j] == 0 && !visitado[i][j])
63             {
64                 mappa(mapa, visitado, filas, columnas, i, j);
65                 regiones++;
66             }
67         }
68     }
69
70     cout << "Mapa ingresado:\n";
71     for (int i = 0; i < filas; ++i)
72     {
73         for (int j = 0; j < columnas; ++j)
74         {
75             cout << mapa[i][j];
76         }
77         cout << endl;
78     }
79     // ELIAZ SEBASTIAN BOBADILLA CAMARENA 20244697E
80
81     cout << "Número de regiones transitables conectadas: " << regiones << endl;
82
83     for (int i = 0; i < filas; ++i)
84     {
85         delete[] mapa[i];
86         delete[] visitado[i];
87     }
88     delete[] mapa;
89     delete[] visitado;
90
91     return 0;
92 }
93

```

```

● → maxe g++ problema_tres_mapa.cpp
● → maxe ./a.out
Ingrese el número de filas y columnas del mapa: 4 5
Ingrese los valores del mapa (0 = camino, 1 = obstáculo):
1 0 0 1 1
1 0 1 1 0
1 1 1 0 0
0 1 1 1 1
Mapa ingresado:
10011
10110
11100
01111
Número de regiones transitables conectadas: 3

```

cuatro

```

problema_cuatro_cadenas.cpp > main()
1 // ELIAZ SEBASTIAN BOBADILLA CAMARENA 20244697E
2
3 #include <iostream>
4 using namespace std;
5
6 bool isAlnumChar(char c)
7 {
8     return (c >= '0' && c <= '9') || (c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z');
9 }
10
11 int main()
12 {
13     const int MAXLEN = 1000;
14     char *input = new char[MAXLEN + 1];
15
16     cout << "Ingrese una cadena:\n";
17     cin.getline(input, MAXLEN);
18
19     char *clean = new char[MAXLEN + 1];
20     char *src = input;
21     char *dst = clean;
22     while (*src)
23     {
24         if (isAlnumChar(*src))
25         {
26             *dst++ = *src;
27         }
28         ++src;
29     }
30     *dst = '\0';
31     delete[] input;
32
33     int len = 0;
34     for (char *p = clean; *p; ++p)
35     {
36         ++len;
37     }

```

```

char *left = clean;
char *right = clean + (len - 1);
while (left < right)
{
    char tmp = *left;
    *left++ = *right;
    *right-- = tmp;
}

int index = 0;
for (int chunk = 1; index < len; ++chunk)
{
    int remain = len - index;
    int take = (chunk < remain ? chunk : remain);
    for (int k = 0; k < take; ++k)
    {
        cout << clean[index + k];
    }
    cout << '\n';
    index += take;
}

delete[] clean;
return 0;
}

```

```

→ maxe g++ problema_cuatro_cadenas.cpp
● → maxe ./a.out
Ingrese una cadena:
Hola, mundo feliz!
z
il
efo
dnum
aloH

```